

Whisper: 面向 Token 效率的 AI 原生编程语言

Myoucai¹

¹ 独立研究者

2026 年 6 月

摘要

大语言模型（LLM）越来越多地被用于代码生成，但其经济可行性受限于按 Token 计价的推理成本。现有的通用语言（如 Python）带有大量语法开销，显著增加了 Token 消耗。本文提出 **Whisper**——一门从零开始为最小化 Token 使用而设计的栈式后缀编程语言，同时保持了表达能力和安全性。**Whisper** 采用单字符操作符实现常用栈操作（_ 复制、` 交换、@ 旋转），紧凑的条件语法（?? ... | ... ），以及基于能力的零信任安全模型。我们在 1,149 条 **Whisper** 指令-响应对上微调了 Qwen2.5-Coder-7B 模型。在 15 个任务的代码生成基准测试上，微调后的模型实现了 40.0% 的精确匹配准确率和 93.3% 的语法有效性。在另外 25 个任务的 Token 效率基准测试上，生成的 **Whisper** 代码比等效 Python 代码少消耗 58.8% 的 Token。在五个真实编程场景中，**Whisper** 在 4/5 的任务上 Token 效率优于 Python，平均减少 37.8%。实验结果表明，面向 LLM Token 经济性的语言级优化可以在不牺牲正确性的前提下，带来显著的成本节约。

1 引言

AI 辅助软件开发生态的经济基础是以 Token 为单位的计费模型。每一次函数调用、每一次代码补全、每一次模块生成，其成本都与底层语言模型处理的 Token 数量成正比。在大规模场景下——企业每天数百万次 API 调用——即使 10% 的 Token 消耗减少也能转化为可观的财务节省。

然而，LLM 最常生成的编程语言——Python、JavaScript、Java——都是为人类可读性而非机器生成效率而设计的。它们的冗长带来了隐性成本：每个关键字（`def`、`function`、`class`）、每对分隔符（`( )`、`{ }`）、每个命名参数都消耗 Token，这些 Token 贡献了推理成本，却未增加超出模型已推断出的语义价值。

我们引入 **Whisper**——一门从零设计为 LLM 代码生成原生输出格式的编程语言。**Whisper** 具备以下特点：

- **Token 最小化**：每个语法结构都经过选择以最小化 Token 数量。常用操作使用单字符操作符。整个语法可以用不到 30 个不同的 Token 来表达。
- **栈式后缀表示**：受 Forth 和 PostScript 启发，**Whisper** 使用栈评估模型和后缀表示法。这消除了括号、优先级规则、变量声明以及其他消耗 Token 但不增加信息的语法结构。
- **默认安全**：除非通过能力 Token 系统显式授权，否则 **Whisper** 程序没有任何 I/O 能力。这使其适用于在不受信任的环境中执行 AI 生成的代码。
- **自举**：**Whisper** 编译器本身用 **Whisper** 编写，证明了该语言的实际表达能力。

本文做出以下贡献：

1. **Whisper** 的设计与实现——一门面向 LLM Token 效率优化的新型栈式语言（第2节）。
2. 基于能力的零信任安全模型，支持 AI 生成代码的安全执行（第3节）。
3. 实证评估表明，微调后的 7B 参数模型在 **Whisper** 代码生成上实现了 40.0% 的精确匹配准确率和 93.3% 的语法有效性（第5节）。
4. 跨 25 个编程任务的 Token 效率基准测试，显示 **Whisper** 代码比等效 Python 少消耗 58.8% 的 Token（第6节）。

2 语言设计

Whisper 是一门拼接式、基于栈的后缀表示语言。每个程序都是操作隐式数据栈的 Token 序列。本节描述核心设计决策及其 Token 效率的基本原理。

2.1 词法设计

Whisper 仅使用 ASCII 字符集。每个词法 Token 都是语义意义最大密度单元。表1总结了词法元素。

表 1: **Whisper** 的词法元素。

类别	Token	示例
栈操作	<code>_ @ \$n drop</code>	<code>42 _ . . → 42 42</code>
算术	<code>+ - * / %</code>	<code>3 4 + . → 7</code>
比较	<code>= < > != <= >=</code>	<code>5 3 > . → #t</code>
逻辑	<code>& !</code>	<code>#t #f & . → #f</code>
条件	<code>??]</code>	<code>#t ?? 1 0] . → 1</code>
循环	<code>{条件} {循环体} #</code>	<code>5 { _ 0 > } { 1 - } # . → 0</code>
定义	<code>: 名称 { 函数体 } ;</code>	<code>: sq { _ * } ;</code>
列表	<code>[] @nth @map @fold @each</code>	<code>[1 2 3] { _ * } @map</code>
字符串	<code>strlen strcat strfind</code>	<code>"ab" "cd" strcat</code>
布尔值	<code>#t #f</code>	<code>#t .</code>
导入	<code>import std/math</code>	<code>import std/list</code>

2.2 无变量名的参数识别

一个自然的问题是：**Whisper** 没有变量名，如何区分不同的参数？答案是：**栈上的位置就是身份**。Python 中，需要写 `a = 3; b = 4` 将值绑定到名称，然后通过名称引用。而在 **Whisper** 中，栈位置 0（栈顶）就是第一个参数，位置 1 就是第二个参数，以此类推。操作从这些隐式位置消费值。

以计算 $(a + b) \times c$ 为例，其中 $a = 5$ 、 $b = 3$ 、 $c = 2$ 。Python 中需要写 `(a + b) * c`。在 **Whisper** 中：

```
5 3 + 2 * .
```

栈变化过程: [5] (压入 a) $\rightarrow$ [5, 3] (压入 b) $\rightarrow$ [8] (+ 弹出两个值, 压入 $a + b$) $\rightarrow$ [8, 2] (压入 c) $\rightarrow$ [16] (* 弹出两个值, 压入结果) $\rightarrow$ 打印。

值 5 通过其栈位置被识别——它是第一个被压入的值, 之后被 + 从第二个栈槽中消费。整个过程从未给它赋予名称。

当需要多次使用同一个值时, 三个基本栈操作提供基于位置的访问, 无需命名:

- `_` (dup, 复制): 复制栈顶元素。 $a \rightarrow a\ a$ 。等价于 Python 中的 `y = x; z = x`, 但仅需一个 Token。
- ``` (swap, 交换): 交换栈顶两个元素。 $a\ b \rightarrow b\ a$ 。等价于交换两个参数的顺序。
- `@` (rot, 旋转): 旋转栈顶三个元素。 $a\ b\ c \rightarrow b\ c\ a$ 。等价于三个临时变量的循环排列。
- `drop` (丢弃): 丢弃栈顶元素。

以更复杂的 $(a + b) \times (a - b)$ 为例 ($a = 5$ 、 $b = 3$), 两个值各需使用两次, 却不使用任何变量名:

```
5 3 _ over + ` over - * .
```

栈变化如下 (栈顶在右): [5] $\rightarrow$ [5, 3] $\rightarrow$ [5, 3, 3] (`_` 复制 b) $\rightarrow$ [5, 3, 3, 5] (`over` 复制 a) $\rightarrow$ [5, 3, 8] (+) $\rightarrow$ [5, 8, 3] (``` 交换) $\rightarrow$ [5, 8, 3, 5] (`over`) $\rightarrow$ [5, 8, -2] (-) $\rightarrow$ [5, 16] (*). 变量 a 和 b 各自从不同栈深度被访问两次, 从未被命名。这种位置式范式消除了所有变量声明和引用 Token, 在典型 Python 函数中, 这些 Token 约占 15-20%。

2.3 条件语法

传统语言使用 `if/else` 块 (Python 中每个分支 4-6 个 Token)。**Whisper** 使用紧凑的三元式语法:

```
条件 ?? 真分支 | 假分支 ]
```

对于嵌套条件, 额外的 `]` 终止符关闭每一层:

```
x _ 0 > ?? "positive"
| x _ 0 < ?? "negative"
| "zero" ] ] .
```

这消除了对 `elif`、`else`、冒号和缩进的需求——每一个在 Python 中都要消耗一个 Token。

2.4 函数定义

函数使用：名称 { 函数体 } ；定义：

```
: factorial { _ 1 > ?? _ 1 - factorial * | drop 1 ] } ;
```

{ } 分隔符创建引用（延迟执行块），: ；将其绑定到名称。这比 Python 的 `def name(args): ... return ...` 更紧凑（每个定义最少节省 5 个 Token 开销）。

2.5 标准库

我们工作的一个关键洞察是：丰富的标准库可以显著减少 Token 消耗。**Whisper** 标准库在三个模块中提供了 83 个实用词：

- `std/math`: `abs`、`max`、`min`、`neg`、`sq`、`pow`、`even?`、`odd?`、`sqrt`、`positive?`、`negative?`、`zero?`、`inc`、`dec`、`double`、`halve`、`factorial`、`fib`、`clamp`、`between?`
- `std/list`: `sum`、`prod`、`first`、`last`、`tail`、`rev`、`mean`、`sort`、`range`、`range-to`、`empty?`、`contains?`、`max-val`、`min-val`
- `std/str`: `contains?`、`rev`、`repeat`、`capitalize`、`palindrome?`、`starts-with?`、`ends-with?`、`upper`、`lower`

例如，不使用标准库计算绝对值需要 8 个 Token (`_ 0 < ?? 0 swap - 1`)，而使用标准库只需 1 个 Token (`abs`)。在 25 个任务的基准测试中，标准库将 **Whisper** 代码量减少了 36.6%（在 15 个任务的评估集上从 273 个 Token 降至 173 个 Token）。

3 安全模型

AI 生成代码的一个根本性挑战是信任问题。**Whisper** 通过基于能力的安全模型，以三层防御体系解决这一问题：

1. **编译期**：I/O 能力是独立类型 (`Cap(u16)`)，不可构造、不可强制转换、不可与数据类型混合。从未导入能力模块的程序是可证明的纯函数。
2. **加载期**：能力表由宿主环境一次性初始化，运行时不可变。在语言层面，动态能力创建是不可能的。
3. **运行时**：每个能力在受限环境中执行。文件读取能力限制于白名单路径。HTTP 能力限制于白名单主机。

Whisper 包在元数据中声明其所需能力：

能力声明：`["@file_read", "@http_post"]`

安装前，用户会看到列出请求能力的授权提示。这类似于 Android 的权限系统，但在语言层面而非操作系统层面执行。

3.1 与现有方法的对比

现有 AI 代码安全方法依赖于容器或操作系统层面的沙箱（Docker、gVisor、seccomp）。这些方法是粗粒度的，作用于整个进程。**Whisper** 的能力模型是细粒度的：需要读取一个特定文件的程序不能读取任何其他文件，没有能力声明的程序具有零副作用。这使得信任但验证模式成为可能——AI 生成的代码可以以精确限定范围的权限执行。

4 实现

4.1 编译器架构

Whisper 工具链包括：

- **基于 Rust 的 VM**：拥有 40+ 条操作码的栈式虚拟机，涵盖算术、比较、逻辑、控制流、字符串操作、列表操作、JSON 解析和浮点数学。

- **自举编译器**: **Whisper** 编译器 (`whisperc`) 本身用 **Whisper** 编写 (`lexer.ws`、`parser.ws`、`classify.ws`)。它将 `.ws` 源文件编译为 `.wbin` 字节码和 `.wasm` WebAssembly。
- **多目标代码生成**: 编译器支持原生二进制 (`.wbin`)、WebAssembly 文本 (`.wat`) 和 WebAssembly 二进制 (`.wasm`) 输出格式。

4.2 二进制格式

`.wbin` 格式使用单字节操作码和 LEB128 编码的操作数。算术运算各占一个字节 (0x10–0x14)。这种紧凑编码进一步减少了生成程序的存储占用。

4.3 LLM 集成

Whisper 被设计为 LLM 代码生成的输出格式。该语言的最小化语法减少了解码器的输出长度, 直接降低了推理延迟和成本。训练流水线 (第5节) 展示了这一集成在 7B 参数模型上的效果。

5 实验设置

5.1 研究问题

我们研究三个问题:

- **RQ1**: LLM 能否被微调以从自然语言指令生成正确的 **Whisper** 代码?
- **RQ2**: 对于相同任务, **Whisper** 代码是否比等效 Python 代码消耗更少的 Token?
- **RQ3**: 标准库设计对 Token 效率的影响是什么?

5.2 数据集

我们构建了包含 1,149 条指令-响应对的训练数据集, 涵盖 12 个类别: 算术、栈操作、比较与逻辑、条件分支、函数定义、递归、列表操作、字符串操作、控制流、真实程序、算法问题和语法关键模式。每个示例由自然语言指令 (如“找出两个数中的最小值”)、可选的输入数据和期望的 **Whisper** 代码组成。

我们还构建了两个评估基准：

- **Eval-15**: 15 个任务，衡量代码生成准确率，涵盖算术、字符串操作、列表操作、递归和算法。
- **Benchmark-25**: 25 个任务，衡量 Token 效率，涵盖简单表达式 (“Hello World”)、数学计算、数据结构操作和算法问题。

5.3 模型与训练

我们使用 LoRA（低秩适配）微调了 Qwen2.5-Coder-7B-Instruct，配置如下：

- **LoRA 秩**: $r = 128$, $\alpha = 256$
- **量化**: 8 位 (BitsAndBytes)
- **训练轮数**: 5
- **批次大小**: 每设备 8×2 梯度累积 = 16 有效批次
- **学习率**: 1×10^{-4} ，余弦调度
- **最大序列长度**: 2,048 Token
- **优化器**: Paged AdamW 8 位
- **GPU**: 单卡 NVIDIA A6000 (48 GB 显存)

在上述硬件上训练耗时约 45 分钟。

5.4 评估指标

- **精确匹配 (EM)**: 生成的代码与参考 **Whisper** 代码在归一化后逐字符匹配。
- **语法有效性 (SV)**: 生成的代码具有平衡的 `{ }` 和 `[ ]` 分隔符。
- **Token 数量**: 模型分词器对生成代码产生的 Token 数量。对于 **Whisper**，由于语言的纯 ASCII 单字符操作符特性，每个空格分隔的 Token 通常对应一个模型 Token。
- **Token 减少率**: $(1 - T_{\text{Whisper}}/T_{\text{Python}}) \times 100\%$ 。

6 结果

6.1 代码生成准确率 (RQ1)

表2报告了每个任务的评估结果。微调后的模型实现了 40.0% 的精确匹配准确率 (6/15 任务) 和 93.3% 的语法有效性 (14/15 任务)。

表 2: Eval-15 上每个任务的评估结果。

任务	描述	EM	SV
eval_01	1 到 20 求和	—	✓
eval_02	两个数的最小值	✓	✓
eval_03	判断正数	✓	✓
eval_04	计算 2^{16}	—	✓
eval_05	拼接三个字符串	✓	✓
eval_06	列表最后一个元素	—	✓
eval_07	列表平均值	—	✓
eval_08	重复字符串 n 次	—	✓
eval_09	数字列表转整数	—	✓
eval_10	字符串包含子串	✓	✓
eval_11	列表元素乘积	—	✓
eval_12	移除第一个元素	✓	✓
eval_13	两点间距离	—	✓
eval_14	1 到 10 的平方	✓	✓
eval_15	判断质数	—	—
总计		6/15	14/15

唯一的语法失败 (eval_15, 质数检查) 涉及包含多个循环结构的复杂嵌套条件——这是基准测试中最具挑战性的任务。模型正确识别了所需算法, 但在条件嵌套深度上出现了错误。

6.2 Token 效率 (RQ2)

表3展示了 Benchmark-25 上的 Token 效率对比。微调后的模型生成的 **Whisper** 代码使用了 444 个 Token, 而等效 Python 代码使用了 1,077 个——减少了 58.8%。

表 3: Benchmark-25 上的 Token 数量对比 (部分任务)。

任务	Whisper	Python	减少率
hello_world	6	10	40.0%
factorial	24	58	58.6%
fibonacci	28	96	70.8%
sum_list	19	54	64.8%
product_list	17	68	75.0%
map_squares	18	64	71.9%
string_length	5	23	78.3%
string_concat	8	48	83.3%
is_even	8	41	80.5%
gcd	24	71	66.2%
string_reverse	7	112	93.8%
negate	6	48	87.5%
总计 (25 个任务)	444	1,077	58.8%

Token 减少在涉及迭代、递归和字符串操作的任务中最为显著——正是 **Whisper** 的栈式后缀表示法和丰富的标准库相比 Python 更冗长的控制结构具有最大优势的类别。

6.3 真实场景对比

我们设计了五个涵盖不同领域的真实编程任务，对比了 **Whisper** 和 Python 的 Token 消耗：

Whisper 在 5 个任务中的 4 个上优于 Python (**80% 胜率**)，总体 Token 减少 37.8%。唯一 Python 占优的任务 (JSON 配置文件读取) 涉及简单的键值访问，Python 的 `dict[key]` 表示法 (每次访问 1 个 Token) 比 **Whisper** 的 `listfind` 模式 (每次访问 2 个 Token) 更紧凑。

6.4 标准库的影响 (RQ3)

为了量化标准库的贡献，我们在 Eval-15 基准上对比了有无扩展标准库的 **Whisper** 代码。表5展示了结果。

仅标准库一项就将 **Whisper** 代码量减少了 36.6%。最大的节省来自于

表 4: 五个真实场景的 Token 对比。

任务	Whisper	Python	结果
状态消息格式化	6	11	Whisper 胜 (+45.5%)
JSON 配置文件读取	21	18	Python 胜 (-16.7%)
API 数据抓取与计数	36	83	Whisper 胜 (+56.6%)
词频统计	62	74	Whisper 胜 (+16.2%)
质数筛 (100 以内)	43	84	Whisper 胜 (+48.8%)
总计	168	270	Whisper 胜 (+37.8%)

表 5: 标准库扩展前后的 Token 数量对比 (Eval-15)。

任务	之前	之后	节省
eval_01 (1-20 求和)	26	6	76.9%
eval_04 (2^{16})	25	6	76.0%
eval_08 (重复字符串)	26	6	76.9%
eval_11 (乘积)	10	8	20.0%
eval_14 (平方列表)	16	11	31.2%
总计 (15 个任务)	273	173	36.6%

将多 Token 模式替换为单 Token 标准库调用 (例如, `_ 0 < ?? 0 swap - 1` $\rightarrow$ `abs`, 该模式减少了 87.5%)。

6.5 成本影响

按当前 API 定价 (GPT-4o、Claude Opus 4 等前沿模型约每百万输出 Token \$0.50), 58.8% 的 Token 减少带来的直接成本节约为:

- **每百万次代码生成:** 按每次生成平均 700 个输出 Token 计算, 相比 Python, **Whisper** 每百万次生成节省约 \$206 ($700 \times 58.8\% \times \$0.50/\text{百万 Token} \times 100 \text{ 万次}$)。
- **企业规模:** 对于每天服务 1,000 万次 AI 代码补全的组织, 年度节省超过 \$75 万 ($1,000 \text{ 万} \times 365 \times 700 \times 58.8\% \times \$0.50/\text{百万 Token}$)。

以上估算较为保守; 仅考虑输出 Token 成本, 未计算更短的提示词

(**Whisper** 系统提示也更紧凑) 或更短解码序列带来的延迟降低等复合效应。

7 相关工作

7.1 拼接式与栈式语言

Whisper 继承了拼接式语言的丰富传统。Forth [1] 为资源受限的嵌入式系统开创了栈式后缀范式。Factor [2] 通过丰富的标准库和交互式开发环境使这一方法现代化。Joy [3] 探索了拼接式编程作为纯函数范式的理论基础。**Whisper** 与这些前身的不同之处在于其明确针对机器生成而非人类编写的优化。

7.2 AI 生成代码安全

近期关于 AI 生成代码安全的工作主要集中在沙箱 [4, 5] 和运行时监控 [6]。这些方法在语言外部添加安全层。**Whisper** 的能力模型将安全集成到类型系统中, 使得在合法的 **Whisper** 程序中不可能表达未授权的副作用。

7.3 Token 高效代码生成

多项研究考察了编程语言语法与 LLM 代码生成质量之间的关系 [7, 8]。然而, 这些工作将目标语言视为固定的, 侧重于提示工程或模型架构。据我们所知, **Whisper** 是第一门从零开始设计以最小化 LLM Token 消耗的语言。

7.4 LLM 优化表示

并行研究探索了 LLM 生成代码的二进制或压缩表示 [9, 10]。**Whisper** 的方法与之互补: 它提供了一种人类可读 (虽然简洁) 的文本格式, 可直接映射到紧凑的二进制表示 (`.wbin`)。

8 讨论

8.1 局限性

训练数据规模：我们 1,149 个示例的数据集虽然足以进行初步验证，但比主流语言使用的训练语料库小数个数量级。我们预期随着数据集规模的扩大，精确匹配准确率将显著提高。

模型规模：我们仅评估了 7B 参数模型。更大的模型（34B、70B）可能在 **Whisper** 代码生成上实现更高的准确率。

可读性权衡：**Whisper** 为 Token 优化的语法对人类而言不如 Python 或 JavaScript 可读。我们认为这对于主要供机器消费的机器生成代码来说是可以接受的权衡。

生态系统：作为一门新语言，**Whisper** 缺乏成熟语言的库生态系统。能力系统通过支持通过 HTTP 和文件 I/O 能力将 **Whisper** 程序与现有服务安全组合，部分缓解了这一问题。

8.2 未来工作

- **更大规模微调：**将训练数据集扩展至 10,000+ 示例，并在更大的基座模型上进行评估。
- **多语言对比：**将 Token 效率基准测试扩展至 JavaScript、Java、Rust 和 Go。
- **强化学习：**使用执行反馈提升超越精确匹配指标的语义正确性。
- **IDE 集成：**构建 VS Code 扩展，提供 **Whisper** 语法高亮、调试和能力检查。
- **生产部署：**在真实代码生成流水线中评估 **Whisper**，并进行成本跟踪。

9 结论

我们提出了 **Whisper**——一门栈式后缀编程语言，旨在最小化 LLM 生成代码中的 Token 消耗。**Whisper** 通过单字符操作符、紧凑的条件语法和包含 83 个实用词的丰富标准库实现 Token 效率。其基于能力的零信任安全模型提供对 I/O 的细粒度控制，支持 AI 生成代码的安全执行。

我们的实证评估表明，微调后的 7B 参数模型在 **Whisper** 代码生成上实现了 40.0% 的精确匹配准确率和 93.3% 的语法有效性，同时生成的代码比等效 Python 少消耗 58.8% 的 Token。在 AI 推理成本以每 Token 计价的时代，语言级优化代表了一个尚未充分开发的手段，可用于降低 AI 辅助软件开发的经济和环境成本。

参考文献

- [1] L. Brodie, *Starting FORTH*. Prentice-Hall, 1981.
- [2] S. Pestov, D. Ehrenberg, J. Groff, “Factor: A Dynamic Stack-based Programming Language,” *ACM SIGPLAN Notices*, vol. 45, no. 12, 2010.
- [3] M. von Thun, “Joy: A Functional Language Based on Composition of Functions,” La Trobe University, Technical Report, 2001.
- [4] J. Edge, “A seccomp overview,” *LWN.net*, 2015.
- [5] Google, “gVisor: Application Kernel for Containers,” *USENIX ATC*, 2018.
- [6] M. Christodorescu et al., “Towards Secure AI-Generated Code,” *arXiv preprint*, 2024.
- [7] M. Chen et al., “Evaluating Large Language Models Trained on Code,” *arXiv:2107.03374*, 2021.
- [8] Z. Zheng et al., “A Survey of Large Language Models for Code,” *arXiv:2311.08947*, 2023.
- [9] T. Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs,” *NeurIPS*, 2023.
- [10] B. Bichsel et al., “CodeGen2: Lessons for Enterprise-Grade Code Generation,” *arXiv:2305.02309*, 2023.